TalentScout Hiring Assistant - Complete Implementation Guide

Executive Summary

This document provides a comprehensive guide for building an intelligent Hiring Assistant chatbot using LLMs, Streamlit, and modern prompt engineering techniques. The system is designed for "TalentScout," a fictional recruitment agency specializing in technology placements.

1. System Architecture & Design

1.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│              USER INTERFACE (Streamlit)              │
│  - Chat display with message history              │
│  - Input field for user messages                  │
│  - Session management                             │
└──────────────────────────────────────────────────────────────────────┘
                          │
                          │
        ┌─────────────────┼─────────────────┐
        │                 │                 │
    ┌───▼────┐      ┌────▼────┐      ┌────▼────┐
    │ Prompts  │    │ Utils      │    │  Config  │
    │ Module   │    │ Module     │    │  Module  │
    └────┬────┘      └────┬────┘      └────┬────┘
        │                 │                 │
        └─────────────────┼─────────────────┘
                          │
        ┌─────────────────▼─────────────────┐
        │    Conversation Manager        │
        │  - Context tracking            │
        │  - Flow orchestration          │
        │  - State management            │
        └─────────────────┬─────────────────┘
                          │
        ┌─────────────────▼─────────────────┐
        │  Large Language Model (LLM)    │
        │  - OpenAI GPT-3.5/4            │
        │  - Context-aware responses     │
        │  - Technical question gen      │
        └─────────────────┬─────────────────┘
                          │
        ┌─────────────────▼─────────────────┐
        │    Data Handler Module         │
        │  - Privacy compliance          │
        │  - Secure storage              │
```

```
          │   - Anonymization                │
          └──────────────────────────────────┘
```

1.2 Conversation Flow

Stage 1: Greeting & Introduction

- Chatbot greets candidate
- Explains purpose of the interview
- Sets expectations

Stage 2: Information Gathering

- Name collection
- Email address
- Phone number
- Years of experience
- Desired position(s)
- Current location
- Tech stack declaration

Stage 3: Technical Assessment

- Generate 3-5 questions based on tech stack
- Ask questions progressively
- Evaluate responses
- Maintain context

Stage 4: Closing

- Summarize collected information
- Thank candidate
- Explain next steps
- Offer graceful exit

2. Prompt Engineering Strategy

2.1 Core Prompting Principles

Clarity & Specificity

- Define exact task requirements
- Use structured formats (JSON, lists)
- Provide clear examples

Context Management

- Include conversation history
- Maintain candidate profile
- Track interview stage

Chain-of-Thought Reasoning

- Break complex tasks into steps
- Guide LLM through reasoning process
- Ensure consistent outputs

Constraint Implementation

- Define boundaries and limitations
- Specify response formats
- Prevent deviation from purpose

2.2 Master System Prompt

```
You are TalentScout, an intelligent recruiting assistant for a technology
recruitment agency. Your role is to conduct initial screening interviews
with technology candidates.

**Core Responsibilities:**
1. Gather essential candidate information
2. Generate tailored technical questions based on their tech stack
3. Maintain professional, friendly conversation
4. Never deviate from your purpose as a recruiting assistant
5. Handle candidate information with privacy and respect

**Interview Stages:**
- Stage 1: Greeting (Be warm and welcoming)
- Stage 2: Information Gathering (Collect systematically)
- Stage 3: Technical Assessment (Evaluate proficiency)
- Stage 4: Closing (Thank and next steps)

**Constraints:**
- Keep responses concise (2-3 sentences per message)
- Ask one question at a time
- Don't ask irrelevant questions
- If unclear input received, ask for clarification politely
- Use conversational, professional tone
- Exit gracefully if candidate uses termination keywords
```

2.3 Specialized Prompts

Information Gathering Prompt

```
Extract and structure the following information from the conversation:
{
  "candidate_name": "string",
  "email": "string (validate email format)",
  "phone": "string",
  "years_experience": "number",
  "desired_positions": ["array of positions"],
  "location": "string",
  "tech_stack": ["array of technologies"],
  "interview_stage": "enum: greeting|gathering|assessment|closing"
}

Current conversation context:
{conversation_history}

Only include fields that have been explicitly mentioned. If critical information
is missing, guide the conversation to collect it. Ensure data privacy and
validation standards.
```

Technical Question Generation Prompt

```
Based on the candidate's declared tech stack, generate 3-5 technical questions
to assess their proficiency.

**Candidate Profile:**
- Tech Stack: {tech_stack}
- Experience Level: {years_experience} years
- Target Position: {desired_position}

**Question Generation Guidelines:**
1. Tailor difficulty to experience level
2. Mix theoretical and practical questions
3. Focus on core concepts and best practices
4. Ask progressively challenging questions
5. Format as clear, unambiguous questions

Generate questions in JSON format:
{
  "questions": [
    {"number": 1, "question": "...", "topic": "..."},
    ...
  ]
}

Ensure questions are:
- Relevant to the declared tech stack
- Appropriate for experience level
- Designed to assess actual proficiency
- Fair and industry-standard
```

Context Preservation Prompt

```
You are continuing a conversation with a candidate. Maintain context about:
- Their profile: {candidate_profile}
- Previous questions asked: {questions_history}
- Their responses: {response_history}
- Current stage: {current_stage}

When responding:
1. Reference previous information to show you're paying attention
2. Build on their previous answers
3. Ask follow-up questions if responses are incomplete
4. Transition smoothly between stages
5. Keep the conversation natural and flowing

Current message from candidate: "{user_message}"

Respond in a way that maintains context and continues the interview professionally.
```

3. Implementation Details

3.1 Session State Management

```
# Initialize session state
if "conversation_history" not in st.session_state:
    st.session_state.conversation_history = []
    st.session_state.candidate_profile = {}
    st.session_state.interview_stage = "greeting"
    st.session_state.questions_asked = []
    st.session_state.responses_collected = []
    st.session_state.message_count = 0
```

3.2 Information Gathering Flow

```
INTERVIEW_STAGES = {
    "greeting": {
        "next_stage": "gathering",
        "fields_to_collect": [],
        "prompt": "greeting_prompt"
    },
    "gathering": {
        "next_stage": "assessment",
        "fields_to_collect": [
            "candidate_name", "email", "phone", "years_experience",
            "desired_positions", "location", "tech_stack"
        ],
        "prompt": "gathering_prompt"
    },
    "assessment": {
        "next_stage": "closing",
```

```
        "fields_to_collect": [],
        "prompt": "assessment_prompt"
    },
    "closing": {
        "next_stage": None,
        "fields_to_collect": [],
        "prompt": "closing_prompt"
    }
}
```

3.3 Data Privacy Handling

```
class PrivacyHandler:
    """Ensures GDPR/data privacy compliance"""

    @staticmethod
    def anonymize_data(email):
        """Convert email to anonymized ID"""
        return hashlib.sha256(email.encode()).hexdigest()[:12]

    @staticmethod
    def validate_email(email):
        """Validate email format"""
        pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
        return re.match(pattern, email) is not None

    @staticmethod
    def secure_store(candidate_data):
        """Store data with privacy considerations"""
        # Simulate secure storage
        # In production: use encrypted database + audit logs
        return {
            "anonymized_id": PrivacyHandler.anonymize_data(candidate_data["email"]),
            "profile": candidate_data,
            "timestamp": datetime.now().isoformat(),
            "gdpr_compliant": True
        }
```

4. Technical Question Generation

4.1 Tech Stack Mapping

```
TECH_STACK_QUESTIONS = {
    "Python": [
        "What are list comprehensions, and when would you use them?",
        "Explain the difference between shallow and deep copy in Python.",
        "How does Python's memory management work? What is garbage collection?"
    ],
    "JavaScript": [
        "Explain event delegation and why it's useful.",
        "What's the difference between let, const, and var?",
```

```
        "How do closures work in JavaScript?"
    ],
    "React": [
        "What are React hooks, and why were they introduced?",
        "Explain the difference between state and props.",
        "What is the virtual DOM, and why is it important?"
    ],
    "Django": [
        "Explain Django's MTV architecture.",
        "What is middleware in Django?",
        "How do Django's ORM query optimizations work?"
    ],
    # ... more technologies
}
```

4.2 Dynamic Question Selection

```
def generate_technical_questions(tech_stack, experience_level):
    """Generate appropriate technical questions"""

    questions = []
    difficulty_map = {
        "junior": "easy",
        "mid-level": "medium",
        "senior": "hard"
    }

    difficulty = difficulty_map.get(experience_level, "medium")

    for tech in tech_stack:
        tech_questions = TECH_STACK_QUESTIONS.get(tech, [])
        if tech_questions:
            # Select questions based on difficulty
            selected = select_by_difficulty(tech_questions, difficulty)
            questions.extend(selected)

    return questions[:5]  # Return max 5 questions
```

5. Handling Edge Cases & Fallback Mechanisms

5.1 Unrecognized Input Handling

```
def handle_unrecognized_input(user_message, context):
    """Handle unclear or off-topic inputs gracefully"""

    fallback_prompts = {
        "off_topic": "I appreciate that, but I'm specifically here to help with your tech
        "unclear": "I didn't quite understand. Could you rephrase that? {}",
        "spam": "Let's keep our conversation focused on the interview. {}"
    }
```

```
    # Classify the input
    input_type = classify_input(user_message)

    # Provide appropriate response
    fallback_response = fallback_prompts.get(input_type, fallback_prompts["unclear"])
    return fallback_response.format(get_context_specific_prompt(context))
```

5.2 Context Loss Recovery

```
def recover_context(conversation_history):
    """Recover and reconstruct context from history"""

    candidate_info = extract_candidate_info(conversation_history)
    questions_asked = extract_questions(conversation_history)
    current_stage = determine_current_stage(candidate_info)

    return {
        "candidate_profile": candidate_info,
        "questions_history": questions_asked,
        "current_stage": current_stage,
        "missing_fields": identify_missing_fields(candidate_info)
    }
```

6. Deployment Considerations

6.1 Local Deployment

Command:

```
streamlit run app.py
```

Access: http://localhost:8501

6.2 Cloud Deployment (AWS)

Using AWS EC2 + Streamlit Cloud:

```
# Push to GitHub repository
git push origin main

# On Streamlit Cloud dashboard:
# 1. Connect GitHub account
# 2. Select repository
# 3. Configure environment variables
# 4. Deploy (automatic on push)
```

6.3 Environment Variables

```
# .env file
OPENAI_API_KEY=your_api_key_here
MODEL_NAME=gpt-3.5-turbo
STREAMLIT_SERVER_PORT=8501
STREAMLIT_SERVER_ADDRESS=localhost
GDPR_COMPLIANCE=true
```

7. Testing Strategy

7.1 Unit Tests

```
def test_email_validation():
    assert PrivacyHandler.validate_email("test@example.com") == True
    assert PrivacyHandler.validate_email("invalid-email") == False

def test_tech_stack_parsing():
    tech_stack = parse_tech_stack("Python, JavaScript, React")
    assert "Python" in tech_stack
    assert len(tech_stack) == 3

def test_context_extraction():
    history = [{"role": "assistant", "content": "..."}]
    context = extract_context(history)
    assert context["candidate_name"] is not None
```

7.2 Conversation Flow Tests

```
def test_full_conversation_flow():
    """Simulate complete interview"""
    assert stage == "greeting"
    send_message("Hello")
    assert stage == "gathering"
    send_candidate_info()
    assert stage == "assessment"
    # ... continue testing
```

8. Challenges & Solutions

Challenge 1: Context Retention

Problem: LLM loses context in long conversations
Solution:

- Implement conversation summarization

- Use sliding window of last 10 messages
- Store extracted candidate profile in session state

Challenge 2: Hallucination Prevention

Problem: LLM generates false technical questions
Solution:

- Use curated question banks
- Implement temperature=0.3 for consistency
- Validate questions against tech stack

Challenge 3: Data Privacy

Problem: Sensitive candidate data exposure
Solution:

- Anonymize email addresses
- Use simulated storage (no real DB)
- Implement GDPR compliance checks
- Add audit logging

Challenge 4: Handling Diverse Tech Stacks

Problem: Many technologies, need relevant questions
Solution:

- Create comprehensive tech stack mapping
- Use LLM for custom question generation
- Validate questions before presenting

Challenge 5: Multi-Turn Context Management

Problem: Maintaining coherent conversation flow
Solution:

- Store candidate profile in session state
- Track interview stage explicitly
- Use context preservation prompts
- Implement fallback mechanisms

9. Performance Optimization

9.1 Caching Strategy

```
@st.cache_resource
def initialize_llm_client():
    """Cache LLM client to avoid re-initialization"""
    return OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

@st.cache_data
def get_tech_stack_questions():
    """Cache technical questions database"""
    return TECH_STACK_QUESTIONS
```

9.2 API Call Optimization

```
def chat_with_gpt_streaming(prompt, temperature=0.7):
    """Stream responses for better UX"""
    client = initialize_llm_client()

    with st.spinner("Thinking..."):
        stream = client.chat.completions.create(
            model="gpt-3.5-turbo",
            messages=prompt,
            stream=True,
            temperature=temperature
        )

        # Display streaming response
        st.write_stream(format_stream_response(stream))
```

10. Future Enhancements

10.1 Bonus Features (High Priority)

1. **Sentiment Analysis**
    - Gauge candidate emotions
    - Detect stress or confusion
    - Adjust questioning accordingly
2. **Multilingual Support**
    - Auto-detect candidate language
    - Support 5+ languages
    - Maintain professionalism across languages
3. **Personalized Responses**

- Track user history
- Customize based on preferences
- Provide tailored feedback

10.2 Advanced Features (Future)

1. **Video Interview Integration**
   - Capture video responses
   - Analyze body language
   - Automatic transcription
2. **Resume Parsing**
   - Extract candidate info from resume
   - Auto-populate profile
   - Validate against stated experience
3. **Automated Scoring**
   - Rate candidate responses
   - Calculate proficiency scores
   - Generate hiring recommendations

11. Conclusion

This Hiring Assistant represents a practical application of LLM technology in recruitment. By combining careful prompt engineering, robust session state management, and thoughtful error handling, we create a professional tool that automates initial candidate screening while maintaining a positive candidate experience.

Key Takeaways

✅ **Prompt Engineering:** Clear, structured prompts guide LLM behavior
✅ **Context Management:** Session state enables coherent multi-turn conversations
✅ **Data Privacy:** GDPR-compliant handling of sensitive information
✅ **Scalability:** Modular architecture supports easy extension
✅ **User Experience:** Professional interface with intuitive interactions

Appendix: Quick Reference

Installation:

```
pip install -r requirements.txt
```

Running the App:

```
streamlit run app.py
```

Key Files:

- `app.py` - Main application
- `prompts.py` - Prompt templates
- `config.py` - Configuration
- `data_handler.py` - Privacy & storage
- `utils.py` - Utility functions

Environment Setup:

```
cp .env.example .env
# Edit .env with your OpenAI API key
```

Document Version: 1.0
Last Updated: November 2024
Created for: TalentScout AI/ML Intern Assignment

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]

⁂

1. https://discuss.streamlit.io/t/best-practices-for-building-genai-apps-with-streamlit/113364
2. https://www.linkedin.com/pulse/personal-chatbot-interface-elasticsearch-streamlit-han-xiang-choong-iciuc
3. https://www.reddit.com/r/learnmachinelearning/comments/1kpx308/llm_interviews_prompt_engineering/
4. https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state
5. https://discuss.streamlit.io/t/chat-assistant-portfolio-project-streamlit-app/80005
6. https://www.promptingguide.ai
7. https://docs.kanaries.net/topics/Streamlit/streamlit-session-state
8. https://discuss.streamlit.io/t/can-someone-please-explain-example-chatbot-code/81356
9. https://www.gsdcouncil.org/blogs/advanced-prompt-engineering-interview-questions-insights
10. https://discuss.streamlit.io/t/streamlit-session-states-in-users-simultaneous-activity/35996
11. https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/
12. https://palantir.com/docs/foundry/aip/best-practices-prompt-engineering/
13. https://www.youtube.com/watch?v=5l9COMQ3acc
14. https://docs.streamlit.io/get-started/fundamentals/advanced-concepts
15. https://towardsdatascience.com/rapid-prototyping-of-chatbots-with-streamlit-and-chainlit/
16. https://www.datacamp.com/blog/llm-interview-questions
17. https://docs.streamlit.io/develop/concepts/architecture/session-state
18. https://discuss.streamlit.io/t/multi-turn-chatbot-using-streamlit-open-ai-and-own-dataset/51326
19. https://coderpad.io/blog/interviewing/prompt-engineering-for-technical-interview-questions/

20. https://towardsdatascience.com/session-state-for-streamlit-dc9ea1685ea/